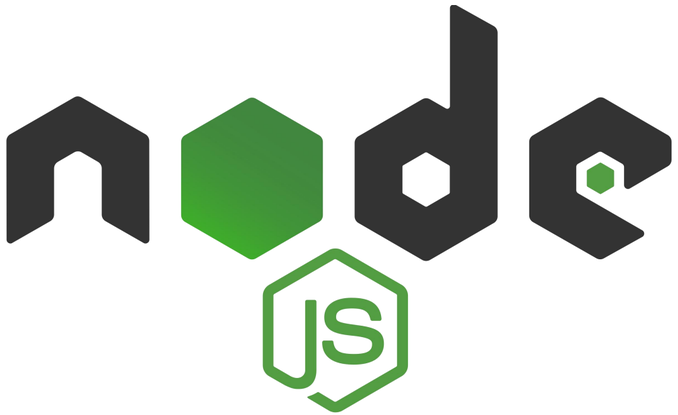
iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

CAPACITAÇÃO EM DESENVOLVIMENTO FULL STACK

Módulo 05: Desenvolvimento Back-end com Node.js e Express
Instrutor: Janei Vieira
Aula 7: Aula 7 - RESTful API



iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Janei Vieira

 Full Stack Developer 🧑‍💻

🎉 24 anos de experiência em tecnologia. Especialista em infraestrutura, automação, containers (Docker/Kubernetes) e desenvolvimento back-end para sistemas comerciais e aplicações em .NET C#, Python e Node.JS

🧑‍💻 Foco em Front-ends com Razor Pages e Streamlit.



Histórico Acadêmico

- **Graduação:** Gestão da Tecnologia da Informação.
- **MBA:** Em Gestão Ágil de Projetos e Processos
- **Especializações:**
  - Redes e Computação Distribuída (**IFMT**).
  - Engenharia DEVOPS (**IFMT**)
  - Big Data (Ciência de Dados).
  - Computação Em Nuvem (Cloud Computing).

Mestrando: Propriedade Intelectual para Transferência de Tecnologia e inovação (**UFRR**)

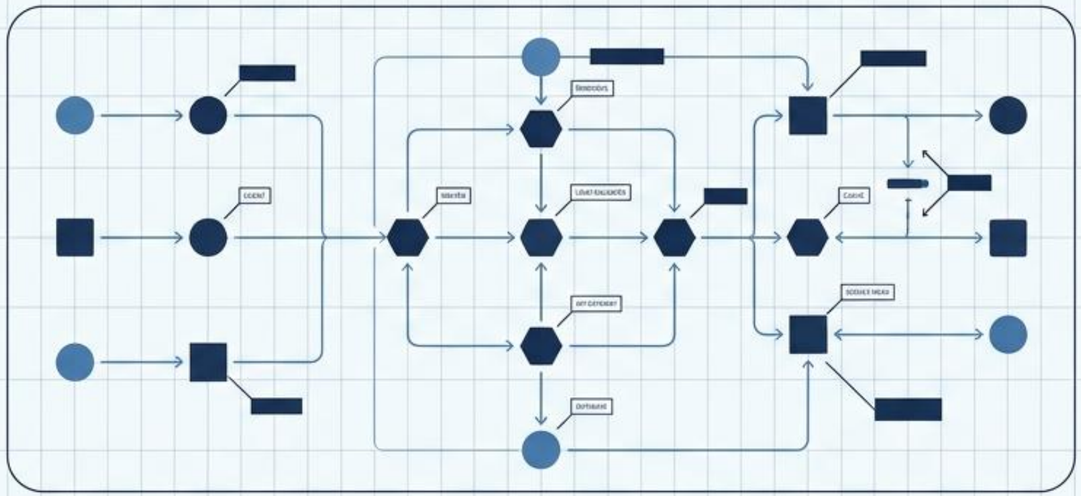
https://lattes.cnpq.br/4083863936027227

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA
Paixão por Desenvolver Talentos

Objetivos da Aula

- **Compreender** os princípios fundamentais da arquitetura REST.
- **Aprender** a utilizar os códigos de status HTTP de forma correta e semântica.
- **Identificar** e aplicar boas práticas no design de APIs RESTful.
- **Refatorar** uma API não RESTFul

Princípios de APIs RESTful



A Arquitetura da Web: Do Conceito à Refatoração

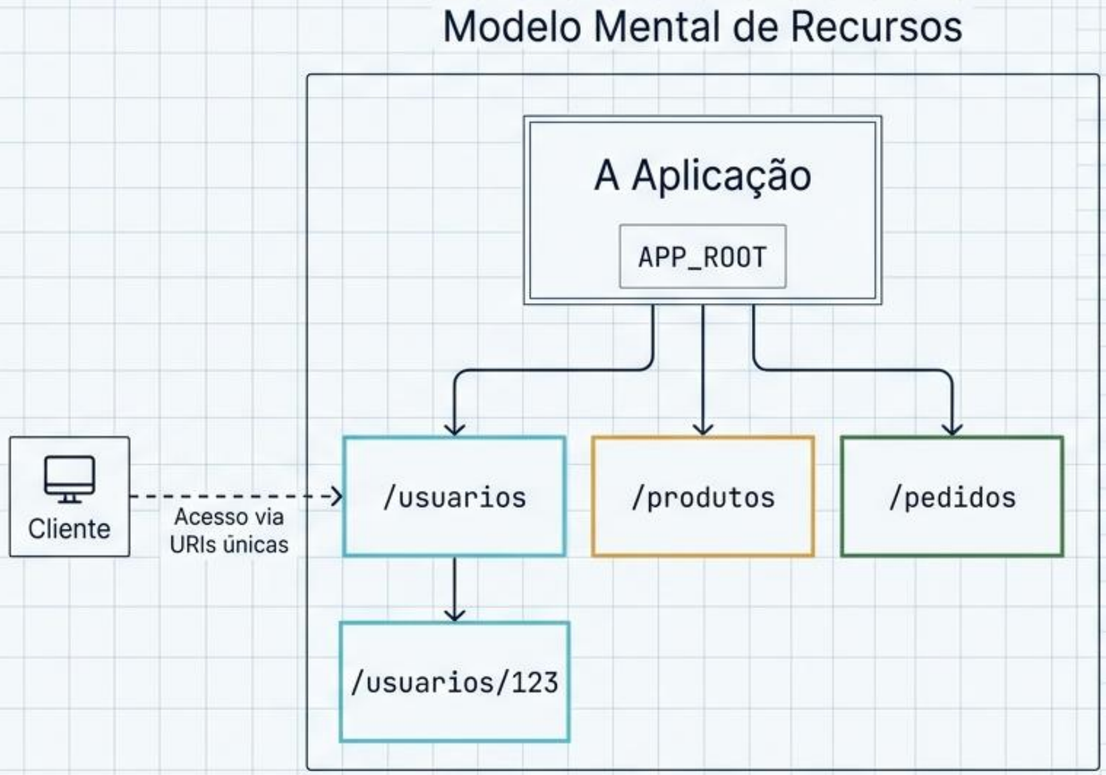
SCALE 1:100

DIMENSIONS IN MM

NotebookLM

O que é REST?

REST (Representational State Transfer) não é um protocolo, mas um conjunto de princípios arquiteturais para aplicações em rede. A ideia central? Tudo é um Recurso.



NotebookLM

Os 4 Pilares da Arquitetura

Cliente-Servidor

Front-end e back-end operam de forma isolada e independente.

Stateless (Sem Estado)

Cada requisição é independente. O servidor não tem 'memória' de sessões



Cacheable

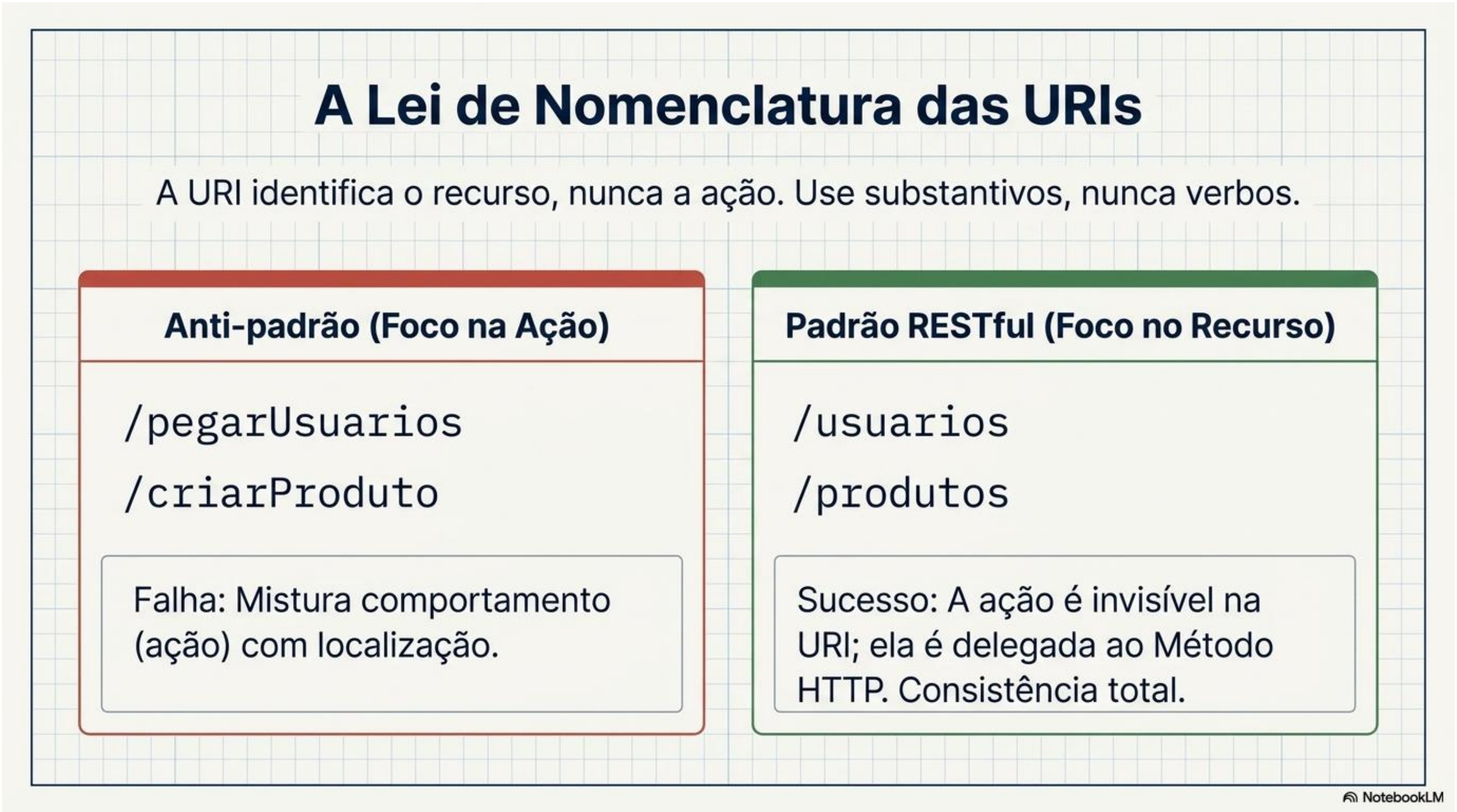
Respostas indicam se podem ser armazenadas em cache, garantindo escalabilidade e performance.

Interface Uniforme

URIs consistentes, respostas autodescritivas (JSON) e HATEOAS (links embutidos para navegação). DELETE/Errors).

NotebookLM


Dissecação de uma Interação RESTful
1
A Ação:
O Método HTTP.
Define o que fazer.
2
A Coleção:
Substantivo no plural.
Define onde buscar.
3
O Identificador Específico:
Localiza o alvo exato na coleção.
GET /usuarios/123
2
Takeaway: O cliente interage com uma representação do recurso (JSON/XML), não com o banco de dados diretamente.
NotebookLM

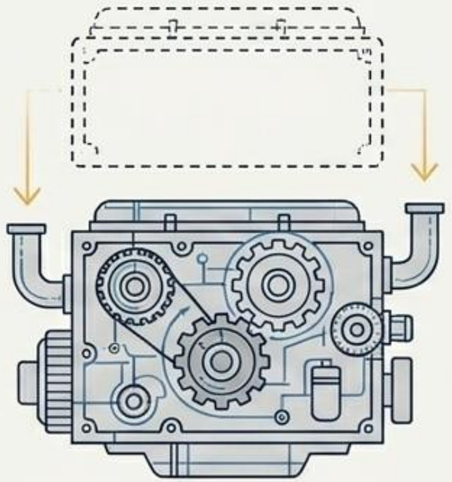

A Lei de Nomenclatura das URIs
A URI identifica o recurso, nunca a ação. Use substantivos, nunca verbos.
Anti-padrão (Foco na Ação)
/pegarUsuarios
/criarProduto
Falha: Mistura comportamento (ação) com localização.
Padrão RESTful (Foco no Recurso)
/usuarios
/produtos
Sucesso: A ação é invisível na URI; ela é delegada ao Método HTTP. Consistência total.
NotebookLM


Métodos HTTP: O Motor de Ação
GET
Ler / Buscar
Operação segura. Não altera altera dados.
Ex: GET /usuarios
POST
Criar
Adiciona um novo recurso.
Ex: POST /usuarios
PUT / PATCH
Atualizar
Modifica um recurso existente.
DELETE
Remover
Exclui o recurso especificado.
Ex: DELETE /usuarios/123
NotebookLM

O Dilema da Atualização: PUT vs. PATCH

PUT (Atualização Completa)

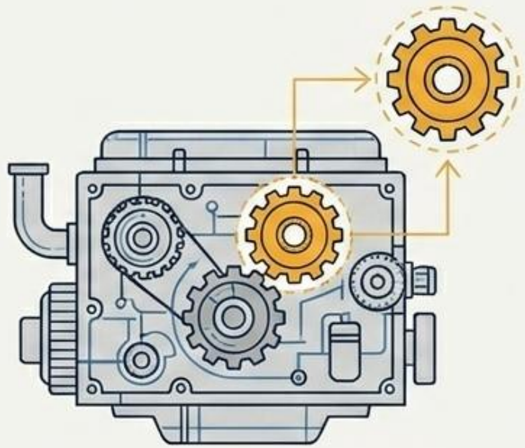
O cliente envia o recurso inteiro. Substituição total.



`PUT /usuarios/123`

PATCH (Atualização Parcial)

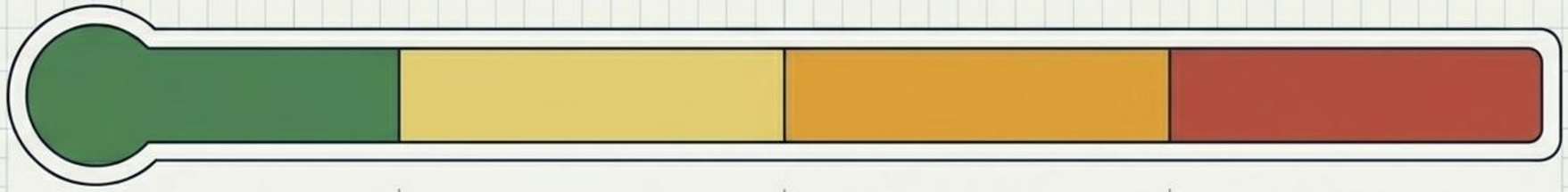
O cliente envia apenas os campos alterados. Eficiência e precisão.



`PATCH /usuarios/123`

NotebookLM

Códigos de Status: O Idioma do Servidor



Faixa 2xx (Sucesso)

- Requisição recebida, entendida e aceita.

Faixa 3xx (Redirecionamento)

- Ações adicionais são necessárias.

Faixa 4xx (Erro do Cliente)

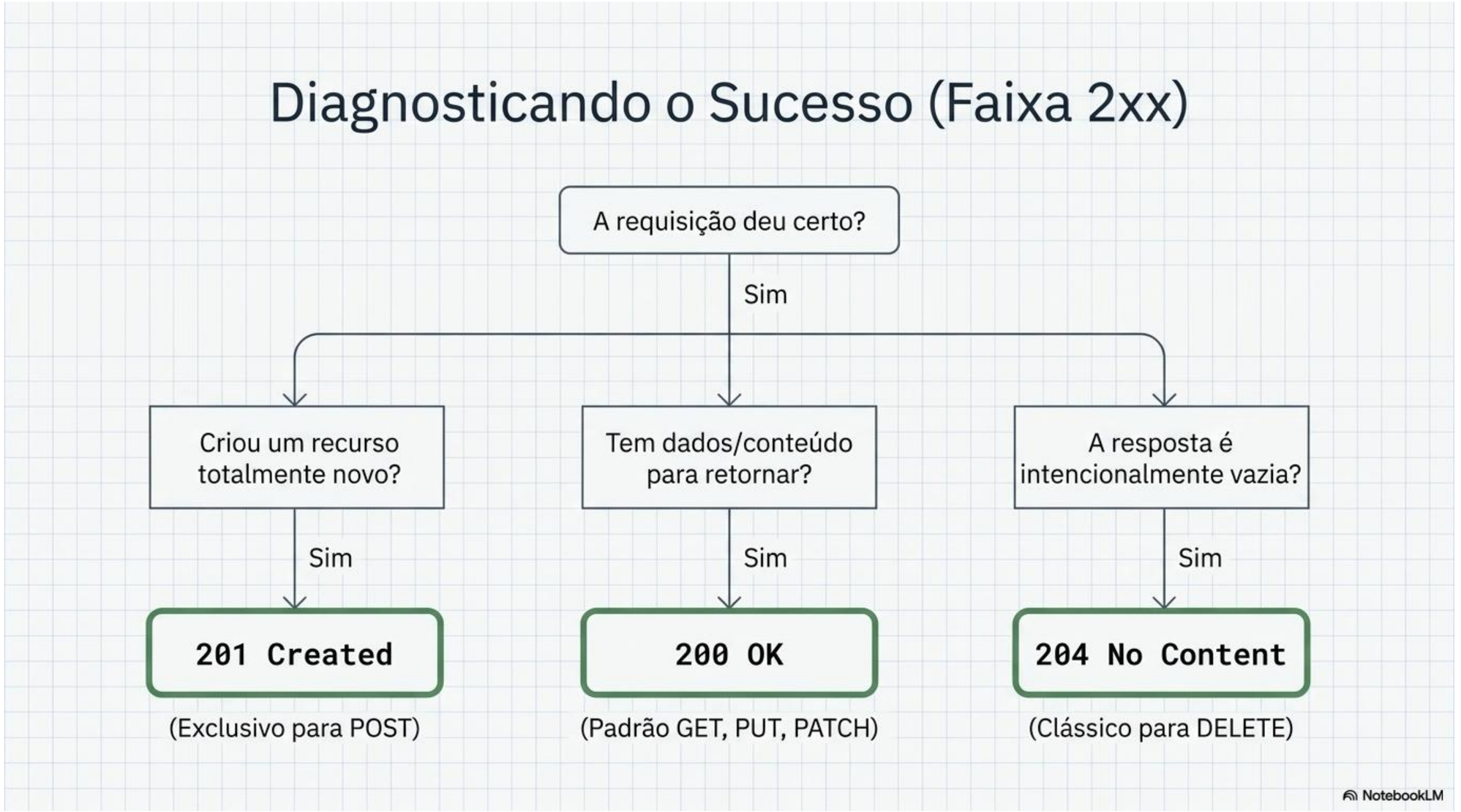
- Sintaxe inválida ou problema na requisição.

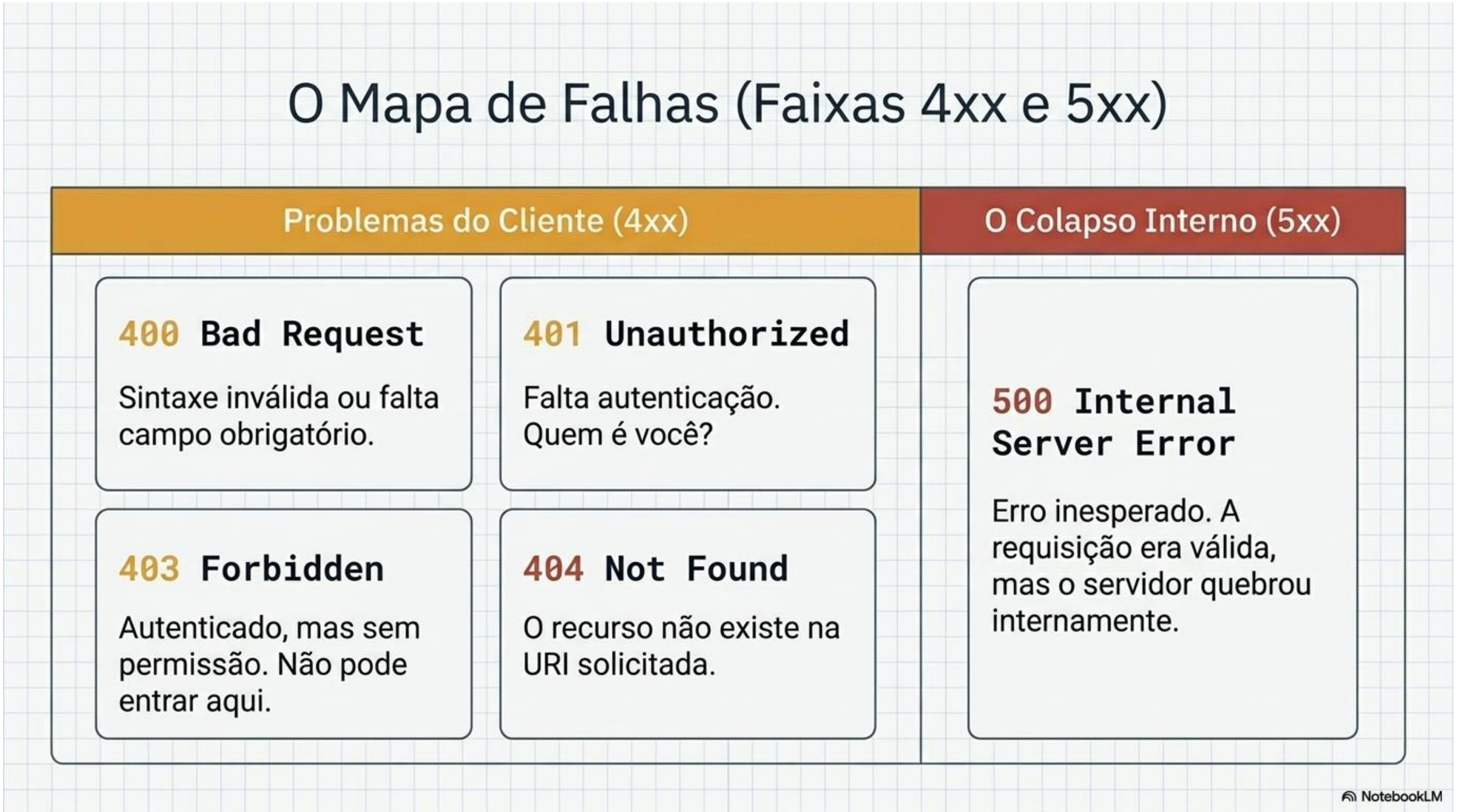
Faixa 5xx (Erro do Servidor)

- Falha ao atender uma requisição válida.

Nota: O servidor informa o resultado. Usar a faixa correta é essencial para a saúde da API.

NotebookLM


Diagnosticando o Sucesso (Faixa 2xx)
A requisição deu certo?
Sim
Criou um recurso totalmente novo?
Tem dados/conteúdo para retornar?
A resposta é intencionalmente vazia?
Sim
Sim
Sim
201 Created
200 OK
204 No Content
(Exclusivo para POST)
(Padrão GET, PUT, PATCH)
(Clássico para DELETE)
NotebookLM


O Mapa de Falhas (Faixas 4xx e 5xx)
Problemas do Cliente (4xx)
O Colapso Interno (5xx)
400 Bad Request
Sintaxe inválida ou falta campo obrigatório.
401 Unauthorized
Falta autenticação. Quem é você?
403 Forbidden
Autenticado, mas sem permissão. Não pode entrar aqui.
404 Not Found
O recurso não existe na URI solicitada.
500 Internal Server Error
Erro inesperado. A requisição era válida, mas o servidor quebrou internamente.
NotebookLM

Laboratório Prático: O Ponto de Partida

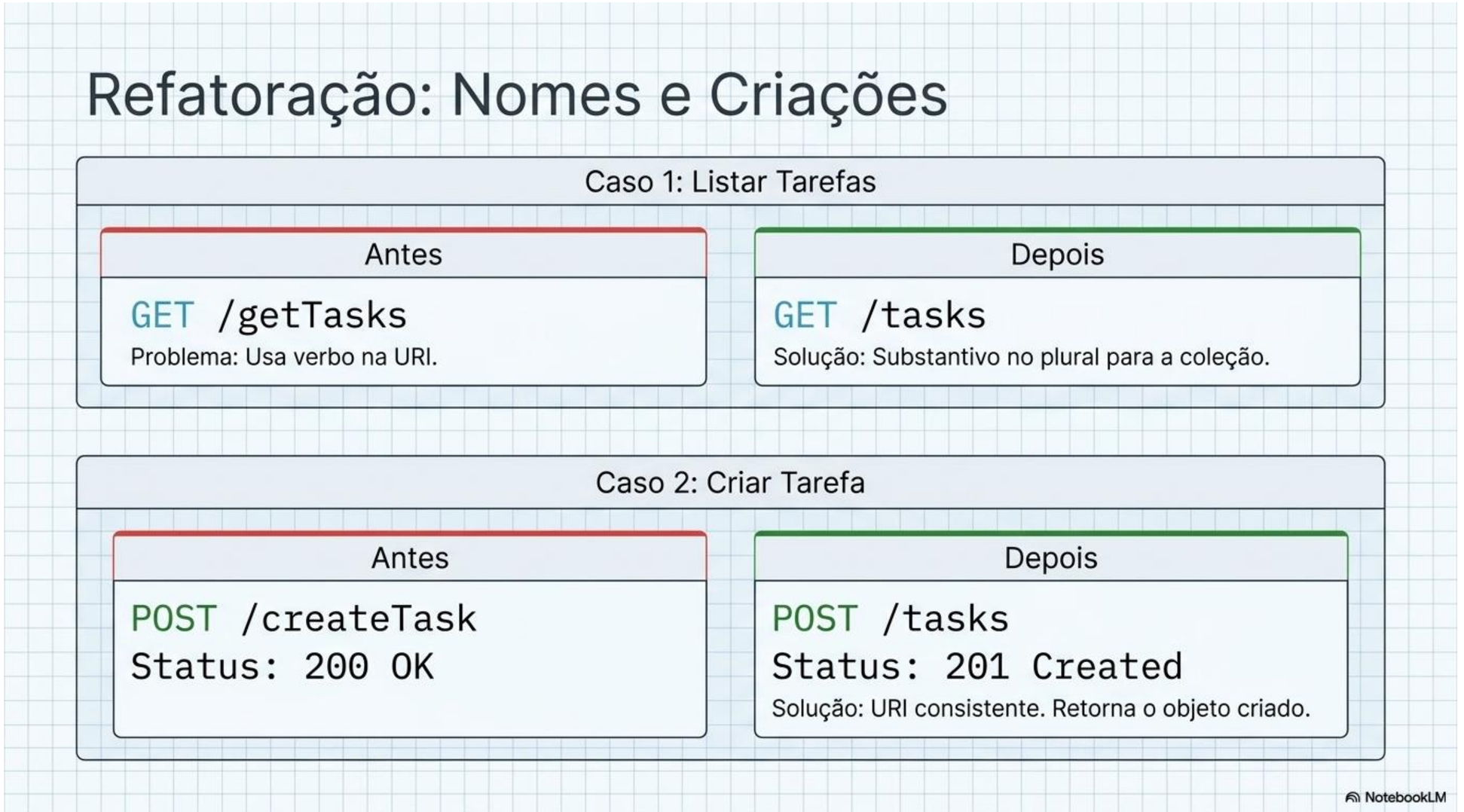
Cenário: Uma API funcional, porém não-RESTful. A arquitetura atual está quebrada.

O Sistema Legado

- ⚠ `GET /getTasks`
- ⚠ `GET /getTaskById?id=1`
- ⚠ `POST /createTask (Retorna 200 OK)`
- ⚠ `POST /updateTask`
- ⚠ `GET /deleteTask?id=1`

Missão: Refatorar endpoint por endpoint para o padrão arquitetural.

NotebookLM


Refatoração: Nomes e Criações
Caso 1: Listar Tarefas
Antes
GET /getTasks
Problema: Usa verbo na URI.
Depois
GET /tasks
Solução: Substantivo no plural para a coleção.
Caso 2: Criar Tarefa
Antes
POST /createTask
Status: 200 OK
Depois
POST /tasks
Status: 201 Created
Solução: URI consistente. Retorna o objeto criado.
NotebookLM

Refatoração: Atualização, Deleção e Parâmetros

Antes - Ruim

Usa query parameters (?id=1) e verbos errados.

```
GET /getTaskById?id=1

POST /updateTask

GET /deleteTask?id=1
```



Perigosamente destrutivo via GET

Depois - RESTful

Usa path parameters (/:id) e métodos HTTP consistentes.

- Buscar: `GET /tasks/:id` (Retorna 404 se não existir)
- Atualizar: `PUT /tasks/:id` (Substitui o recurso)
- Deletar: `DELETE /tasks/:id` (Retorna `204 No Content`)

NotebookLM

A Matriz de Refatoração Concluída

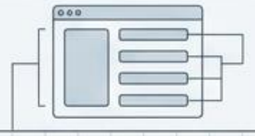
Ação	API Legado (Antes)	API RESTful (Depois)	Status Esperado
Listar Todas	`GET /getTasks`	`GET /tasks`	`200 OK`
Buscar Única	`GET /getTaskById?id=1`	`GET /tasks/:id`	`200 OK (ou 404)`
Criar Nova	`POST /createTask`	`POST /tasks`	`201 Created`
Atualizar	`POST /updateTask`	`PUT /tasks/:id`	`200 OK`
Deletar	`GET /deleteTask?id=1`	`DELETE /tasks/:id`	`204 No Content`

Conclusão: A semântica agora está perfeitamente atrelada à combinação (Verbo + Coleção + ID).

NotebookLM

O Checklist de Ouro RESTful

1 Trate tudo como **Recursos tangíveis**, não como funções soltas.



2 URIs são endereços. Use apenas **substantivos no plural** (ex: `/produtos`).



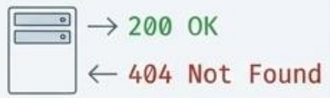
3 Delegue a ação aos **Verbos HTTP** (Deixe o GET, POST, PUT, DELETE trabalharem).



4 **Mantenha a API Stateless.** Cada requisição leva sua própria mochila de dados.



5 **Seja preciso nos Códigos de Status.** O servidor deve falar o idioma da web.



`// Arquitetura limpa é previsível, escalável e universal.`

NotebookLM

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Até a Próxima Aula!

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

node JS